지난 발표 이후의 진행

10/29 계획은 CIC2018 PCAP 재표현이었으나 host 1대 추출의 한계로 CIC2017 전체 망으로 전환

10/29 계획

PCAP에서 인코더로 벡터 생성

충돌·조기 포화를
같은 조건에서 재점검

EXP93 CIC2018 host 1대

날짜당 피해 host 1대만 추출

✕ 정상 모집단이 CSV와 다름
✕ 일부 공격 family 누락
✕ Bot은 피해 host가 아닌 파일

EXP94 CIC2017 전체 망

5일 전체 망 캡처 51 GB

✓ 정상 모집단이 CSV와 동일
✓ CSV와 같은 망에서 비교
✓ 라벨을 세 상태로 재부여

10/29 연구 방향 발표 → **11/02** EXP93 host 1대 → **11/03** CSV 기준 실험 → **11/04** PCAP 실행·비교

CIC2018은 host별 캡처 약 440개/일, 10일 450 GB · CIC2017은 스위치 캡처 하루 1개 파일, 5일 51 GB

host 1대 추출은 비교 조건을 깨뜨려 전체 망 캡처인 CIC2017로 전환

두 view를 두는 이유

가설 CSV의 고정 통계가 정보를 버려 두 문제가 생긴다면, PCAP에서 패킷열로 표현을 만들면 나아진다

CSV (공식 배포본)	view 1 통계 45개	view 2 netFound
flow 정의 CICFlowMeter 원본 (첫 FIN 종료, RST 무시)	flow 정의 고침 (RST·양쪽 FIN·새 SYN·120초)	flow 정의 view 1과 동일
라벨 시간창 + 끝점	라벨 세 상태 (증거 조건)	라벨 view 1과 동일
표현 통계 78개	표현 통계 (CSV와 같은 종류)	표현 패킷 헤더열에서 학습한 벡터

CSV ↔ view 1

flow 정의와 라벨을 고친 효과만 남음
표현은 같은 종류이므로 대조군

view 1 ↔ view 2

표현을 바꾼 효과만 남음
가설이 참이면 여기서 충돌이 줄고 tail이 올라야 함

PCAP에서 벡터까지 여섯 단계

①~④는 두 view가 공유하고, ⑤에서 갈라진 뒤 ⑥에서 같은 평가를 받는다

① PCAP 5개
56.4M 패킷
시각 + 패킷 바이트

② flow로 묶기
같은 두 끝점의 패킷
RST·FIN·SYN·120초 종료

③ 라벨 붙이기
시간창 + 끝점 + 증거
세 상태로 부여

④ 시간순 나누기
앞 70% 학습, 뒤 15% 평가
날짜·family마다

⑤a 통계 계산
모든 패킷에서 45개

view 1
45차원 벡터

두 view가 같은 flow·라벨·분할·표본을 받음

⑤b 토큰화
헤더 필드 → 정수
burst 12 × 패킷 6

netFound 인코더 (고정)
토큰열 → 512차원

PCA 64
학습 데이터로 맞춤

view 2
64차원 벡터

⑥ 평가
충돌 점검 (3층)
oracle 상한

few-shot
XGB · TabPFN
n = 2 ~ 50

공유 단계 ①~④ 위에서 ⑤a 통계와 ⑤b 토큰화·인코더만 다름

① 패킷의 생김새

패킷은 네트워크로 오가는 데이터 조각 하나 · 아래는 월요일 사내 PC가 웹서버로 보낸 실제 패킷 151바이트



이더넷 14 B

바로 옆 장비까지 전달하기 위한 표

보낸 장비 번호
받는 장비 번호

IP 헤더 20 B

어느 컴퓨터로 갈지 적는 칸

보낸 주소 192.168.10.9
받는 주소 69.31.33.224
전체 길이 137 B
TTL 128
프로토콜 6번 (TCP)

TCP 헤더 20 B

어느 프로그램으로 갈지 적는 칸

보낸 포트 1057
받는 포트 80
seq 2039531092
ack 2775650109
신호 ACK+PSH · window 256

payload 97 B

실제 내용

GET /ncsi.txt HTTP/1.1
Connection: Close
User-Agent: Microsoft NCSI
Host: www.msftncsi.com

택배로 비유하면 헤더는 상자 겉의 송장, payload는 상자 속 물건

TTL 패킷이 더 지날 수 있는 장비 수, 운영체제가 시작값을 정함 · window 지금 더 받을 수 있는 양
seq 보낸 데이터의 순번 · ack 어디까지 받았는지 알리는 번호 · TCP 신호는 다음 장들에서 설명

PCAP에 기록되는 것 도착 시각 + 이 바이트 전체

패킷은 겉 정보인 헤더와 속 내용인 payload로 구성되고, 본 연구는 payload 내용을 읽지 않음

① PCAP을 시각순으로 읽고 남기기

PCAP은 스위치를 지나간 패킷을 통째로 녹화한 파일 · 기록 한 건 = 도착 시각 + 패킷 바이트 · 5일치 파일 5개, 51 GB

1 **기록을 한 건씩 읽기**

파일 앞에서 뒤로 56,370,702건을 차례로 읽음

2 **시각순으로 맞추기**

녹화 순서가 뒤바뀐 패킷 33,724건
최근 5초 분량을 모아 두고 시각이 빠른 패킷부터 처리

3 **필요한 패킷만 남기기**

IPv4의 TCP·UDP·ICMP 패킷만 flow로 묶음
나머지는 오른쪽 표처럼 제외

프로토콜 = 주고받는 방식의 약속, IP 헤더의 번호로 구분

구분	패킷 수	설명
전체 기록	56,370,702	5일치
ARP	233,361	IP에 맞는 장비 번호 묻기
IPv6	142,021	이번 구현 범위 밖
기타 장비 신호	37,134	스위치끼리의 신호
조각·기타 프로토콜	2,482	포트를 읽을 수 없음
닫힌 세션의 순수 ACK	113,166	끝난 연결의 마지막 확인
flow에 사용	**55,842,538**	**99.1%**

TCP 번호 6

연결을 맺고 받았는지 확인하며 보냄
웹 · 메일 · 파일 전송
flow 1,424,890개

UDP 번호 17

확인 없이 바로 보냄
도메인 조회 · 시간 동기화
flow 999,509개

ICMP 번호 1

네트워크 상태를 알림
ping · 도달 불가 알림
flow 649개

패킷 56.4M개를 시각순으로 읽어 99.1%를 flow 구성에 사용

② 같은 대화의 패킷을 한 flow로 묶기

flow = 두 컴퓨터의 두 프로그램이 같은 약속으로 주고받은 패킷 묶음, 즉 대화 한 건

IP 주소	포트	프로토콜
네트워크 안의 컴퓨터 주소 192.168.10.9 = 사내 PC 한 대	컴퓨터 안 프로그램의 문 번호 80 웹 · 443 보안 웹 · 53 도메인 조회 접속하는 쪽은 임시 번호 사용	주고받는 방식의 약속 TCP와 UDP는 포트 번호를 따로 사용 같은 53번이어도 다른 통로

다섯 값이 같으면 같은 대화 프로토콜 · IP A · 포트 A · IP B · 포트 B (보낸 쪽과 받는 쪽이 바뀌어도 같음)

패킷	프로토콜	보낸 쪽	받는 쪽	묶인 결과
요청	TCP	192.168.10.9 : 1057	69.31.33.224 : 80	flow A
응답	TCP	69.31.33.224 : 80	192.168.10.9 : 1057	flow A 방향만 반대
인증 접속 1	TCP	192.168.10.9 : 1055	192.168.10.3 : 88	flow B
인증 접속 2	TCP	192.168.10.9 : 1056	192.168.10.3 : 88	flow C 임시 포트가 다름
도메인 조회	UDP	192.168.10.9 : 59285	192.168.10.3 : 53	flow D 프로토콜과 상대가 다름

묶는 이유 패킷 하나로는 행동을 알 수 없고, 대화 한 건 전체를 봐야 요청·응답·종료를 판단 가능

패킷 55.8M개가 같은 규칙으로 flow 2,425,048개가 됨

② TCP 신호와 flow의 끝

→ PC가 보냄 ← 서버가 보냄 · 소속 칸이 각 패킷이 들어가는 flow · 두 대화 모두 월요일 실제 패킷

FIN으로 끝남 · 192.168.10.9:1057 ↔ 웹서버 69.31.33.224:80

#	시각 (μs)	방향	TCP flag	길이	payload	소속
1	0	→	SYN	52	0	flow A
2	19,958	←	SYN+ACK	52	0	flow A
3	20,008	→	ACK	40	0	flow A
4	20,083	→	ACK+PSH	137	97	flow A
5	40,027	←	ACK	40	0	flow A
6	40,640	←	ACK+PSH	219	179	flow A
7	40,643	←	ACK+FIN	40	0	flow A
8	40,685	→	ACK	40	0	flow A
9	40,689	→	ACK+FIN	40	0	flow A
10	**60,574**	**←**	**ACK**	**40**	**0**	**flow A 마감**

RST로 끝남 · 192.168.10.51:38488 ↔ 웹서버 209.15.36.31:80

#	시각 (μs)	방향	TCP flag	길이	payload	소속
1	0	→	SYN	60	0	flow B
2	42,246	←	SYN+ACK	60	0	flow B
3	42,350	→	ACK	52	0	flow B
4	5,079,690	→	ACK+FIN	52	0	flow B
5	5,084,903	←	ACK+FIN	264	212	flow B
6	**5,084,965**	**→**	**RST**	**40**	**0**	**flow B 마감**
7	5,121,931	←	ACK	52	0	제외, 닫힌 뒤 ACK
8	**5,121,962**	**→**	**RST**	**40**	**0**	**새 flow, 잔여 flow**

PC가 4번 FIN으로 먼저 닫아 도착하는 5·7번마다 RST로 응답

첫 RST에서 마감, 둘째 RST는 패킷 1개짜리 새 flow · 잔여 flow 11.7%

TCP 신호

SYN 연결 요청 · ACK 받았다는 확인
PSH 내용 바로 전달 · FIN 보낼 것이 끝남
RST 연결을 버림, 확인 없이 즉시 종료

RST가 오는 두 경우

닫힌 문으로 연결 요청이 왔을 때 거절
이미 닫은 연결로 패킷이 계속 올 때 중단

flow가 끝나는 때

RST · 양쪽 FIN 뒤 ACK · 같은 끝점의 새 SYN
120초 무통신 · 시작 후 120초
UDP·ICMP는 시간 조건만 사용

FIN 종료는 마지막 ACK까지, RST 종료는 첫 RST까지가 한 flow이고 뒤따르는 RST는 새 flow

⑤b 헤더를 숫자 토큰으로 바꾸는 이유

netFound는 문장을 단어 번호로 읽는 언어 모델과 같은 구조 · 헤더의 칸 하나를 단어 하나처럼 숫자로 변환

패킷	IP 헤더 길이	TOS	전체 길이	IP flag	TTL	TCP flag	window	seq 앞	seq 뒤	ack 앞	ack 뒤	urgent
1번 SYN	20	0	52	2	128	2	8192	0	0	0	0	0
2번 SYN+ACK	20	0	52	2	56	18	29200	0	0	0	1	0
4번 요청	20	0	137	2	128	24	256	0	1	0	1	0

패킷 1개 = 숫자 12개 · 같은 자리는 항상 같은 뜻 · TCP flag 2 = SYN, 18 = SYN+ACK, 24 = ACK+PSH · 이 숫자열이 모델의 입력

왜 숫자인가

모델은 숫자열만 읽음

칸마다 뜻을 고정해야
패킷끼리 비교 가능

왜 두 칸으로 나누나

토큰 하나는 0~65,535

seq·ack는 더 큰 수라
앞·뒤 두 칸으로 분할

왜 상대값인가

원본 seq 2039531091은
무작위 출발값

첫 패킷을 0으로 맞춰
주고받은 양만 남김

왜 IP·포트를 빼나

주소를 외우지 않고
행동 모양을 배우도록
저자가 제외

사전학습도 같은 형식

② flow 하나를 ⑤ 두 view로

앞에서 본 flow(패킷 10개, payload 97/179 B, 60.6 ms)를 두 view의 입력으로 바꾸면

⑤b 첫 패킷(SYN)의 토큰

IP 헤더 길이	20
TOS	0
IP 전체 길이	52
IP flag (DF)	2
TTL (Windows)	128
TCP flag (SYN)	2
window	8192
상대 seq·ack	0 0 0 0
urgent	0

포트·IP·시각 없음 · 패킷 10개 모두 같은 방식

⑤a 통계 45개 중 일부

지속 시간	60,574 µs
보낸·받은 패킷	5 · 5
보낸 길이 평균·최대	61.8 · 137
받은 길이 평균·최대	78.2 · 219
패킷 간격 평균	6,730 µs
SYN·FIN·PSH 수	2 · 2 · 2
초기 window	8192 · 29200
초당 패킷	165
목적 포트	80

사람이 정한 요약, 포트 포함

⑤b burst → netFound



6 burst (방향 · 패킷 수)
같은 방향 10 ms 안의 패킷이 한 burst
간격 0·0·20·20·20·20 ms (1 ms 미만 버림)

netFound (고정)

토큰열 → 512차원 벡터

PCA 64

학습 데이터로만 맞춤 · 분산 98.1%

사전학습 라벨 없는 트래픽의
가려진 헤더 토큰 맞히기

view 1은 사람이 정한 요약(포트 포함), view 2는 헤더 토큰에서 배운 벡터(포트·IP·시각 없음)

② 패킷이 적은 flow가 똑같아지는 이유

같다 = 숫자 토큰, 방향, 바이트, 패킷 수, ms 단위 간격이 전부 일치 · IP·포트·시각은 입력에 없어 달라도 같은 입력

패킷 수	flow 수	서로 다른 입력	같은 입력 %
1	129,117	2,196	99.0
2	531,582	34,845	94.8
3~5	691,512	64,728	92.1
6~20	608,631	555,892	11.1
21 이상	309,401	307,588	0.8

짧은 flow의 칸이 거의 고정값인 이유

TTL 운영체제 기본값 64 · 128
길이 빈 패킷 40, 연결 요청 52 · 60
window 운영체제 기본값, RST는 0
상대 seq·ack 0 또는 1
시간 간격 1 ms 미만은 0

패킷 1개 HTTPS 연결 끝의 RST 49,263개

[20, 0, 40, 2, 64, 4, 0, 0, 0, 0, 0, 0]

패킷 2개 스캔 SYN과 거절 RST 79,925개

[20, 0, 60, 2, 62, 2, 29200, 0, 0, 0, 0, 0]
[20, 0, 40, 2, 64, 20, 0, 0, 0, 0, 1, 0]

패킷 4개 도메인 조회 11,891개

질문 [20, 0, 51, 2, 64, 31] 2번 · 답 [20, 0, 51, 0, 128, 31] 2번
UDP는 칸이 6개, 이름 길이가 같으면 토큰도 같음

패킷이 많아지면 길이와 seq가 쌓여 조합이 갈라짐

겹침 자체는 문제가 아니고, 라벨까지 같으면 해롭지 않음

문제가 되는 것은 라벨이 갈리는 4.86%

패킷이 5개 이하인 flow는 칸이 거의 고정값이라 같은 입력이 많고, IP·포트가 달라도 구별되지 않음

③ 라벨: 시간창·끝점·증거의 세 조건

교정 규칙 53개(CNS 2022)를 그대로 적용 · 증거 조건은 공격마다 다르고 저자가 패킷을 보고 정한 것

공격자 IP ↔ 피해자 IP?

아니오 → 정상

공격 시간창 안?

창 밖 → 정상

공격 행위의 증거?

없음 → 보류

모두 예 → 공격

payload 헤더 뒤의 실제 데이터
RST 상대가 연결을 거부·중단한 표시
포트 제외 저자가 사람 조작으로
확인한 연결의 포트 번호

공격	증거 조건	증거가 없는 예 → 보류
FTP · SSH brute force	공격자 payload ≥ 1 B (로그인 시도)	handshake만 하고 끝난 연결
DoS GoldenEye	payload ≥ 1, RST 없음 또는 5초 이상	요청 전에 끊긴 연결
Slowloris · Slowhttptest	payload ≥ 1, 도구 설정 포트 제외	서버 무응답 구간의 빈 연결
Web Brute Force · XSS	payload ≥ 1 (BF는 패킷 ≥ 21) 브라우저 포트 제외	설정·종료 때의 정적 파일 요청
Bot Ares	master와 payload 교환	master 종료 뒤 SYN → RST 재접속
Portscan	SYN 1개와 무응답 또는 RST (payload 없음이 정상)	해당 없음

결과 flow 2,425,048개 = 정상 1,731,862 · 공격 538,428 · 보류 154,743 (6.4%) · 경계 15

보류는 공격에도 정상에도 넣지 않고 학습·채점에서 제외, 수만 집계
저자 권고: Attempted는 독립 class로 쓰지 말 것

보류의 89%는 세션이 닫힌 뒤 남은 잔여 flow · Hulk 무payload 133,626 / 137,908

시간창과 끝점만 맞고 증거가 없는 flow 154,743개는 보류, 그중 89%는 세션이 닫힌 뒤의 잔여 flow

최종 데이터 구성

분할 전 전체 기준 · CSV는 공식 라벨 행 수, PCAP은 세 상태로 나눈 flow 수 · 행 순서는 PCAP 공격 수 내림차순

family	세부 공격 (PCAP 공격 flow 수)	CSV 행	PCAP 공격	PCAP 보류	채점
benign	월요일 전일 + 공격일 배경	2,273,097	1,731,862	–	○
portscan	금 Portscan 161,202 · 목 NMAP 101,936	158,930	263,138	0	○
dos	Hulk 158,571 · GoldenEye 7,567 · Slowloris 3,859 · Slowhttptest 1,741	252,661	171,738	143,383	○
ddos	LOIC-HTTP	128,027	95,683	1,203	○
bf	FTP 4,021 · SSH 2,961	13,835	6,982	4,090	○
bot	Ares	1,966	736	4,067	○
web	Brute Force 73 · XSS 18 · SQL Injection 13	2,180	104	1,954	○ (test 16)
inf	Communication 32 · CoolDisk 4	36	36	45	× 학습 25
heartbleed	Heartbleed	11	11	0	× 학습 7
합계		**2,830,743**	**2,270,290**	**154,743**	**7 family**

CSV와 PCAP의 수 차이는 flow 정의(RST·FIN 처리)와 보류 분리에서 온다 · 목요일 NMAP 101,936개는 CSV에서 BENIGN, 교정 규칙에서 portscan

잔여 flow 284,600개를 빼면 정상 1,609,980 · 공격 513,414 · 보류 17,039 · 경계 15

보류 154,743개(6.4%)를 빼고 7 family를 채점, inf·heartbleed는 표본 부족으로 제외

문제1 입력이 같아지는 단계

같은 flow를 세 단계의 입력으로 보는 이유 · 어느 단계에서 "입력은 같은데 family가 다른" 쌍이 생기는지 보면 정보가 사라지는 단계가 드러남

L1 flow에서 뽑은 정보 전부	L2 netFound 토큰화 뒤	L3 분류기가 받는 벡터
통계 45개 + 자르기 전 토큰 전부 IP·시각·출발 포트·원본 seq는 없음 → 뽑은 정보로 가르지 못하는 양	12 burst × 6 패킷으로 자르고 간격을 ms 단위로 버림한 뒤 → tokenizer가 버린 정보	view 1 통계 45차원 view 2 PCA 64차원 (CSV 78개 통계도 하나의 L3)

예 둘 다 패킷 1개짜리 RST+ACK, 길이 40 B, payload 0, TTL 128

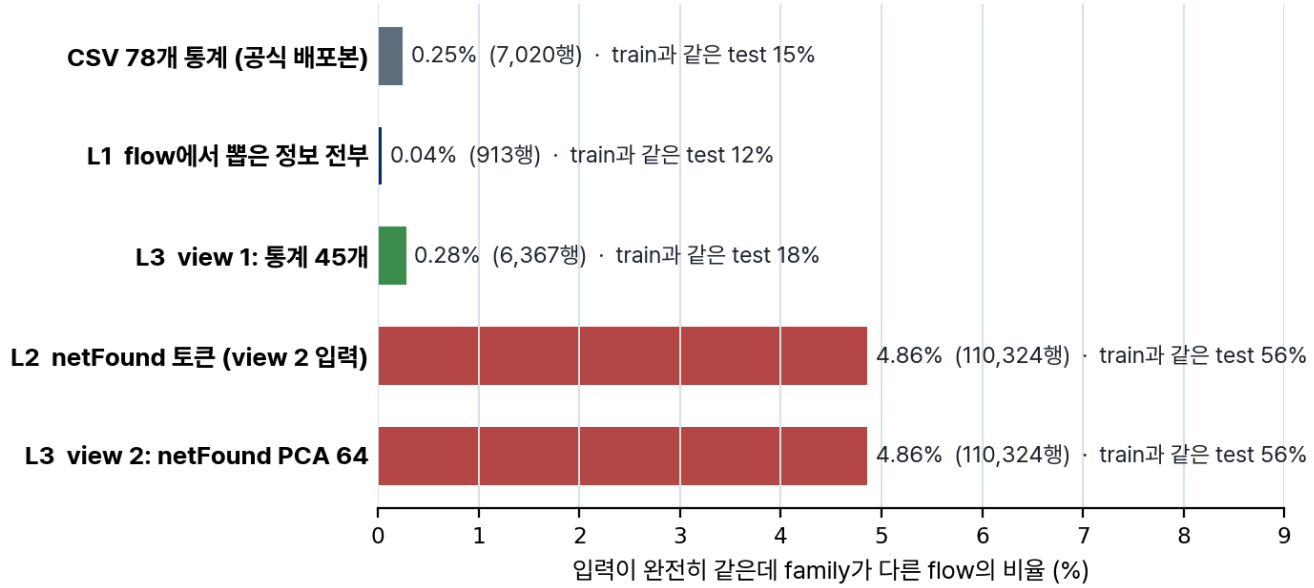
flow	L1 · 통계	L2 · 토큰	L3 · view 1
정상 192.168.10.3:88 → 10.9:1031	목적 포트 1031	[20,0,40,2,128,20,0,0,0,0,0,0]	벡터 다름 (포트)
스캔 응답 10.50:1720 → 10.8:51700	목적 포트 51700	[20,0,40,2,128,20,0,0,0,0,0,0]	벡터 다름 (포트)

L1·통계에서는 포트가 갈라 주지만, 포트가 없는 L2 토큰에서는 두 flow가 완전히 같은 입력 → 정상 11,817 + 스캔 4,363이 한 묶음

L1 0.04% · 통계 45 (view 1) 0.28% = CSV 0.25% 수준 · netFound 토큰 (view 2 입력) 4.86% · 상한(oracle)은 모두 99.8% 이상

문제1 층별 충돌 비율

라벨 있는 flow 2,270,243개 기준 · 막대는 "입력이 완전히 같은데 family가 다른" flow의 비율



충돌의 정체 패킷 1개짜리 RST

세션이 닫힌 뒤 남은 RST가
정상과 스캔 양쪽에서 같은 모양

토큰 [20, 0, 40, 2, TTL, RST, 0, ...]
Linux 정상 49,204 + 스캔 59
Windows 정상 11,817 + 스캔 4,363

포트가 없는 토큰은 TTL로만 구분

L1 flow에서 뽑은 정보로 못 가르는 양 0.04%

L3 통계 45 CSV와 같은 수준 0.28%

netFound 토큰과 PCA 벡터 모두 4.86%

어떤 모델도 넘지 못하는 상한(oracle)

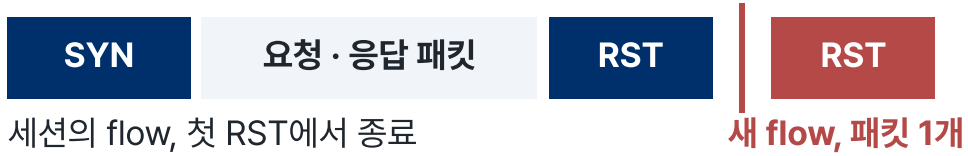
CSV 99.997% · 통계 100% · netFound 토큰 99.79%

충돌이 점수를 깎는 양은 모든 표현에서 작음

현재 flow 규칙에서는 충돌이 남고 netFound에서 CSV보다 커지지만, 상한은 모두 99.8% 이상

문제1 flow 경계가 만든 잔여 flow

RST에서 flow를 바로 닫아 같은 연결의 뒤따르는 패킷이 새 flow가 됨 · 원본 PCAP 헤더로 대조



잔여 flow 판정 기준

직전 세션이 RST·양쪽 FIN으로 닫힌 뒤 2초 안
SYN 없음 · payload 없음 · 패킷 2개 이하

항목	수
잔여 flow	284,600 / 2,425,048
패킷 1개 flow 중 잔여	253,388 / 264,649
보류 중 잔여	137,704 / 154,743
직전 세션과 라벨 일치	146,896 / 146,896
직전 세션과의 간격 중앙값	0.15 ms

구성 같은 쪽 연속 RST 186,820 · 상대의 FIN 29,123

충돌 행 수	현재	잔여 flow 제외
L1 flow에서 뽑은 정보	913	2
L3 통계 45	6,367	69
L2 netFound 토큰	110,324	43,380
보류를 공격으로, 통계 45	138,527	75
보류를 정상으로, L1	131,418	2

남은 netFound 충돌은 거의 전부 스캔 SYN 탐침, 소수 라벨 212행

netFound 원 전처리와의 차이

원 전처리 같은 두 끝점을 새 SYN에서만 분리
RST·FIN·시간 제한으로 닫지 않아 잔여 flow 없음
본 구현 RST 즉시 종료, 잔여 패킷은 순수 ACK만 제거
WTMC 2021이 지적한 종료 패킷 flow가 RST·FIN으로 남음

잔여 flow를 세지 않으면 feature 변경 없이 충돌이 대부분 사라지고 보류 정책 간 충돌 차이도 소멸

문제1 다른 입력과 구별되는 입력

netFound 충돌 묶음의 행에 정보 하나를 더해 train 다수결로 validation을 예측 · test 미사용 · 모델 학습 없는 조회 비교

추가 정보	충돌 행 수	val 정확도 %	잔여 제외 val 정확도 %
없음 (현재 토큰)	110,324	90.4	95.7
목적 포트	1,110	90.4	95.7
서비스 포트 (두 포트 중 작은 값)	12,288	99.2	96.0
원본 seq가 0인지	106,013	99.2	99.4
같은 IP 쌍의 최근 60초 포트 수	12,667	99.8	99.9

목적 포트 충돌 수만 감소

충돌 99% 제거, 정확도 변화 없음
스캔 응답의 목적 포트는
스캐너의 임시 포트

원본 seq 토큰이 지운 정보

패킷 1개 flow의 상대 seq는 항상 0
원본 0이면 닫힌 포트가
SYN에 답한 RST

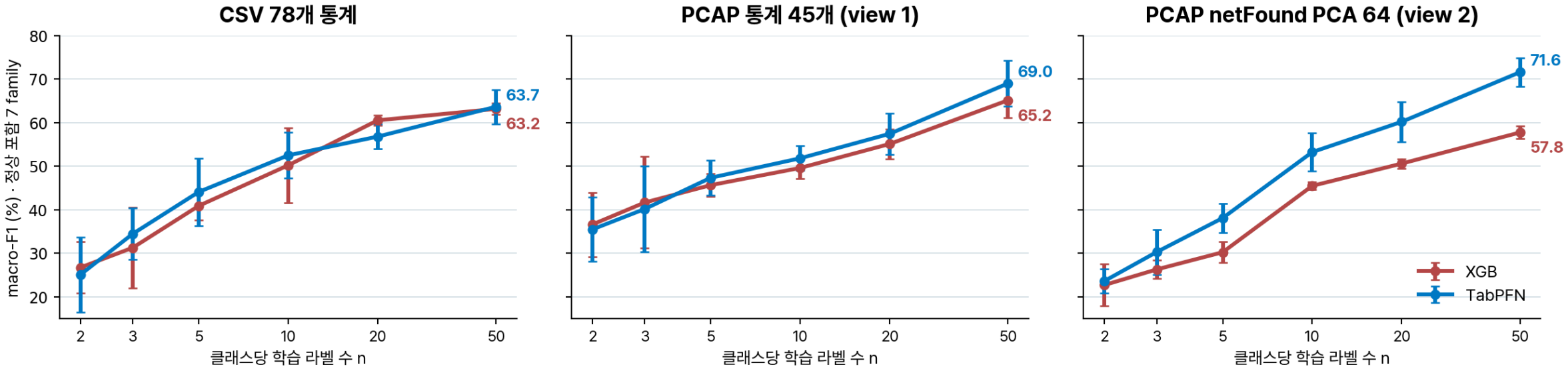
맥락 flow 묶음의 성질

스캔은 flow 하나로 정의되지 않음
스캔 대 정상 단일 feature AUC
0.945에서 0.995로
과거 flow만, IP는 묶는 키로만

충돌 수가 줄어도 구별력이 오르는 것은 아니므로 후보는 validation 정확도로 비교

문제2 라벨 n개로 얼마나 오르나

정상 포함 7 family · class당 라벨 n개 · seed 5개 · 전체 평가 집합 · 두 모델에 같은 행



head 포화는 세 열 동일

공격만 n=20 portscan 98~99.6%
ddos 89~99% · dos 85~98%
n=50까지 1~2 pp 변화

n=50 macro-F1 XGB / TabPFN

CSV 63.2 / 63.7
통계 45 65.2 / 69.0
netFound 57.8 / 71.6

netFound 격차는 비중복 행

학습 350행과 같은 토큰열 test 8.7%
그 행 정확도 두 모델 97.0
나머지 XGB 91.1 · TabPFN 97.5

보류 flow를 어디에 두느냐 (변형 a)

같은 corpus·분할·인코딩에서 보류 154,743개의 라벨만 바꿔 다시 측정 · exclude = 본 실행, attack = CSV 방식, benign = EXP93 방식

보류 정책	L1 충돌	통계 충돌	토큰 충돌	통계 공격만 n=50	통계 정상 n=50	netFound 정상 n=50	bot n=20 통계
exclude (본 실행)	0.04	0.28	4.86	83.5 / 81.4	65.2 / 69.0	57.8 / 71.6	10.4 / 2.9
attack (CSV 방식)	0.04	5.71	4.77	75.1 / 75.8	68.1 / 70.2	57.6 / 66.4	36.8 / 22.1
benign (EXP93 방식)	5.42	5.65	10.15	83.5 / 81.4	63.7 / 68.9	57.1 / 69.0	11.4 / 7.8

충돌은 라벨 있는 flow 대비 % · macro-F1과 class F1은 XGB / TabPFN

attack 통계 충돌 급증

통계 view 충돌 0.28 → 5.71%
그중 dos 130,504행 (Hulk 잔여 flow)
잔여 제외 시 138,527 → 75행
공격만 macro n=50 8 pp 하락
netFound+TabPFN 71.6 → 66.4

benign L1 충돌 급증

L1 정보가 같은데 라벨 다름
0.04 → 5.42% (Hulk 130,500행이 정상)
잔여 제외 시 131,418 → 2행
few-shot 변화 −0.1 ~ −2.6 pp
공격만 과제는 영향 없음

exclude 가장 깨끗

충돌·상한·곡선 모두 최선
세 상태 라벨을 기본 규약으로 유지
잔여 제외 후 보류 17,039개만
host·시간 집계 과제로

보류를 공격으로 두면 통계 충돌이 5.71%로 뛰지만 원인은 잔여 flow, 제외하면 정책 간 충돌 차이 소멸

판정과 다음 단계

문제1 동일 입력·상이 라벨

주원인은 flow 경계의 잔여 flow
잔여 제외 시 통계 0.28 → 0.003%
netFound 토큰 4.86 → 2.04%

상한은 모두 99.8% 이상

문제2 head 조기 포화

세 표현에서 동일
대량·반복 공격의 성질

표현이 바꾸는 것은
tail(bf·bot)과 모델 차이의 부호

남은 과제 희소 class 오탐

bf·bot·web 오탐은 패킷 중앙값
11~109개의 정상 세션, 잔여 flow 0%

잔여 제외 test macro-F1
n=50에서 0.5~1.4 pp 상승

PCAP 전환의 산출물은 새 표현이 아니라 정리된 평가 규약

세 상태 라벨 · 시간순 분할 · 층별 충돌 점검과 상한 · 중복률을 본 연구의 평가 프로토콜로 채택

완료 (a) 보류 → 공격 / 정상 두 정책 비교

중단 (b) PCA 128 · 512

후보 (c) 잔여 flow 제거, 원본 seq 표시, 60초 맥락 feature

하지 않음 CIC2018 전체 450 GB 수집

재집계 기준 문제1의 충돌은 flow 경계 수정으로 대부분 사라지고, 남은 과제는 희소 class의 오탐

부록 n=20 family별 F1

정상 포함 n=20 family별 F1 (%) · 5 seed 평균 · 행 순서는 PCAP 평가 집합 수 내림차순

family	PCAP test	CSV XGB	CSV TabPFN	통계 XGB	통계 TabPFN	netFound XGB	netFound TabPFN
benign	259,782	90.8	89.3	88.9	90.9	89.2	94.9
portscan	39,472	94.2	97.9	84.9	82.3	82.6	89.2
dos	25,761	81.0	75.3	69.2	89.3	73.4	87.6
ddos	14,353	87.5	91.8	89.1	92.7	72.2	94.2
bf	1,048	63.7	26.9	43.0	43.8	32.8	50.7
bot	111	0.0	0.0	10.4	2.9	4.0	4.5
web	16	6.5	16.5	0.4	0.3	0.1	0.3
macro	**340,543**	**60.6**	**56.8**	**55.1**	**57.5**	**50.6**	**60.2**

bot 0.0(CSV)은 평가 집합의 bot이 전부 master 종료 뒤 재접속 flow인 CSV 라벨 결함 · web은 PCAP 평가 집합 16행이라 참고치